

```
const chaveArmazenamento = "vivoSigitmGestores";
```

```
let gestores = JSON.parse(  
  localStorage.getItem(chaveArmazenamento) || "[]"  
);
```

```
let usuarioAtual =  
  localStorage.getItem("vivoSigitmUsuario") || "";
```

```
const elemento = id =>  
  document.getElementById(id);
```

```
function salvarDados() {  
  
  localStorage.setItem(  
    chaveArmazenamento,  
    JSON.stringify(gestores)  
  );  
}
```

```
function mostrarNotificacao(mensagem) {  
  
  const notificacao =  
    elemento("notificacao");  
  
  notificacao.textContent = mensagem;  
  
  notificacao.classList.add("exibir");  
  
  setTimeout(() => {  
    notificacao.classList.remove("exibir");  
  }, 2500);  
}
```

```
function entrarSistema() {  
  
  if (!usuarioAtual) {  
  
    const nome =  
      elemento("nomeUsuario")
```

```

        .value
        .trim();

    if (!nome) {
        alert("Digite seu nome para entrar.");
        return;
    }

    usuarioAtual = nome;

    localStorage.setItem(
        "vivoSigitmUsuario",
        usuarioAtual
    );
}

elemento("usuarioConectado").textContent =
    usuarioAtual;

elemento("loginTela")
    .classList.add("oculto");

elemento("sistema")
    .classList.remove("oculto");

renderizarGestores();
}

function sairSistema() {

    localStorage.removeItem(
        "vivoSigitmUsuario"
    );

    usuarioAtual = "";

    elemento("sistema")
        .classList.add("oculto");

    elemento("loginTela")
        .classList.remove("oculto");

    elemento("nomeUsuario").value = "";

```

```
}
```

```
function abrirModal(indice = -1) {  
  
  elemento("mensagemFormulario")  
    .textContent = "";  
  
  elemento("indiceEdicao").value =  
    indice;  
  
  elemento("tituloModal").textContent =  
    indice < 0  
      ? "Novo gestor"  
      : "Editar gestor";  
  
  elemento("formGestor").reset();  
  
  const estados =  
    elemento("estados");  
  
  [...estados.options].forEach(  
    opcao => opcao.selected = false  
  );  
  
  if (indice >= 0) {  
  
    const gestor =  
      gestores[indice];  
  
    elemento("idGestor").value =  
      gestor.id;  
  
    elemento("nomeGestor").value =  
      gestor.nome;  
  
    elemento("macroGrupo").value =  
      gestor.macroGrupo || "";  
  
    elemento("statusGestor").value =  
      gestor.status;  
  }  
}
```

```

elemento("atuacaoGestor").value =
    gestor.atuacao;

elemento("responsavel").value =
    gestor.responsavel;

elemento("telefone").value =
    gestor.telefone || "";

elemento("email").value =
    gestor.email || "";

elemento("escopo").value =
    gestor.escopo || "";

elemento("atividade").value =
    gestor.atividade || "";

(gestor.estados || []).forEach(
    estado => {

        const opcao =
            [...estados.options]
                .find(
                    item =>
                        item.value === estado
                );

        if (opcao) {
            opcao.selected = true;
        }

    }
);

} else {

    elemento("statusGestor").value =
        "Atuando";

    elemento("atuacaoGestor").value =
        "Internos";

```

```
    elemento("responsavel").value =  
        usuarioAtual;  
}
```

```
    elemento("modalGestor")  
        .classList.remove("oculto");  
}
```

```
function fecharModal() {  
  
    elemento("modalGestor")  
        .classList.add("oculto");  
}
```

```
function escapar(valor) {  
  
    return String(valor ?? "")  
        .replace(  
            /[&<>"/g,  
            caractere => {  
  
                const caracteres = {  
                    "&": "&amp;",  
                    "<": "&lt;",  
                    ">": "&gt;",  
                    "\"": "&quot;",  
                    "'": "&#039;",  
                };  
  
                return caracteres[caractere];  
            }  
        );  
}
```

```
function renderizarGestores() {  
  
    const busca =  
        elemento("buscarId")  
            .value  
            .trim()
```

```
.toLowerCase();
```

```
const status =  
  elemento("filtroStatus")  
    .value;
```

```
const atuacao =  
  elemento("filtroAtuacao")  
    .value;
```

```
const filtrados =  
  gestores  
    .map(  
      (gestor, indice) => ({  
        ...gestor,  
        indice  
      })  
    )  
    .filter(gestor => {  
  
      return (  
  
        (  
          !busca ||  
          String(gestor.id)  
            .toLowerCase()  
            .includes(busca)  
        )  
  
        &&  
  
        (  
          !status ||  
          gestor.status === status  
        )  
  
        &&  
  
        (  
          !atuacao ||  
          gestor.atuacao === atuacao  
        )  
      )  
    }  
  );
```

```
);  
  
});
```

```
elemento("contadorGestores")  
  .textContent =  
    `${filtrados.length} ${  
      filtrados.length === 1  
        ? "registro"  
        : "registros"  
    }`;
```

```
const tabela =  
  elemento("tabelaGestores");
```

```
tabela.innerHTML = "";
```

```
elemento("tabelaVazia")  
  .style.display =  
    filtrados.length  
      ? "none"  
      : "block";
```

```
filtrados.forEach(  
  gestor => {  
  
    const linha =  
      document.createElement("tr");
```

```
    linha.innerHTML = `
```

```
      <td>  
        <span class="id-gestor">  
          ${escapar(gestor.id)}  
        </span>  
      </td>
```

```
      <td>  
        ${escapar(gestor.nome)}  
      </td>
```

</td>

<td>

```
<span
  class="status ${
    gestor.status === "Atuando"
      ? "atuando"
      : "nao-atuando"
  }"
>
  ${escapar(gestor.status)}
</span>
```

</td>

<td>

```
<span class="tag-atuacao">
  ${escapar(gestor.atuacao)}
</span>
</td>
```

<td>

```
${escapar(
  gestor.macroGrupo || "-"
)}
</td>
```

<td>

```
${escapar(
  gestor.responsavel
)}
</td>
```

<td>

```
${escapar(
  (gestor.estados || []).
    .join(", ")
)}
</td>
```

<td>

```
<div class="acoes">
```

```

        <button
            class="acao"
            onclick="abrirModal(${gestor.indice})"
        >
            Editar
        </button>

        <button
            class="acao"
            onclick="excluirGestor(${gestor.indice})"
        >
            Excluir
        </button>

    </div>

</td>
`;

    tabela.appendChild(linha);
}
);
}

```

```

function excluirGestor(indice) {

    const confirmar =
        confirm(
            `Excluir o gestor ${gestores[indice].id}?`
        );

    if (!confirmar) {
        return;
    }

    gestores.splice(
        indice,
        1
    );
}

```

```
);

salvarDados();

renderizarGestores();

mostrarNotificacao(
    "Gestor excluído."
);
}
```

```
elemento("formGestor")
    .addEventListener(
        "submit",
        evento => {

            evento.preventDefault();

            const indice =
                Number(
                    elemento(
                        "indiceEdicao"
                    ).value
                );

            const id =
                elemento(
                    "idGestor"
                ).value
                .trim();

            const duplicado =
                gestores.some(
                    (gestor, indiceAtual) =>

                        String(gestor.id)
                            .toLowerCase() ===
                            id.toLowerCase()

                        &&
```

```

        indiceAtual !== indice
    );

    if (duplicado) {

        elemento(
            "mensagemFormulario"
        ).textContent =
            "Já existe um gestor cadastrado com este ID.";

        return;
    }

```

```

const estados =
    [
        ...elemento(
            "estados"
        ).selectedOptions
    ]
    .map(
        opcao => opcao.value
    );

```

```

if (!estados.length) {

    elemento(
        "mensagemFormulario"
    ).textContent =
        "Selecione pelo menos um estado.";

    return;
}

```

```

const gestor = {

    id,

    nome:
        elemento(

```

```
    "nomeGestor"  
  ).value.trim(),
```

```
macroGrupo:  
  elemento(  
    "macroGrupo"  
  ).value.trim(),
```

```
status:  
  elemento(  
    "statusGestor"  
  ).value,
```

```
atuacao:  
  elemento(  
    "atuacaoGestor"  
  ).value,
```

```
responsavel:  
  elemento(  
    "responsavel"  
  ).value.trim(),
```

```
telefone:  
  elemento(  
    "telefone"  
  ).value.trim(),
```

```
email:  
  elemento(  
    "email"  
  ).value.trim(),
```

```
estados,
```

```
escopo:  
  elemento(  
    "escopo"  
  ).value.trim(),
```

```
atividade:  
  elemento(  
    "atividade"  
  ).value.trim(),
```

```
    atualizadoEm:
      new Date().toISOString()
  };
```

```
if (indice < 0) {
```

```
    gestores.push(
      gestor
    );
```

```
} else {
```

```
    gestores[indice] =
      gestor;
}
```

```
salvarDados();
```

```
fecharModal();
```

```
renderizarGestores();
```

```
mostrarNotificacao(
  indice < 0
    ? "Gestor cadastrado com sucesso."
    : "Gestor atualizado com sucesso."
);
```

```
  }
);
```

```
elemento("entrarSistema")
  .addEventListener(
    "click",
    entrarSistema
  );
```

```
elemento("nomeUsuario")
```

```
.addEventListener(  
  "keydown",  
  evento => {  
  
    if (evento.key === "Enter") {  
      entrarSistema();  
    }  
  
  }  
);
```

```
elemento("sairSistema")  
  .addEventListener(  
    "click",  
    sairSistema  
  );
```

```
elemento("novoGestor")  
  .addEventListener(  
    "click",  
    () => abrirModal()  
  );
```

```
elemento("fecharModal")  
  .addEventListener(  
    "click",  
    fecharModal  
  );
```

```
elemento("cancelarModal")  
  .addEventListener(  
    "click",  
    fecharModal  
  );
```

```
elemento("buscarId")  
  .addEventListener(  
    "input",  
    renderizarGestores
```

```
);
```

```
elemento("filtroStatus")  
  .addEventListener(  
    "change",  
    renderizarGestores  
  );
```

```
elemento("filtroAtuacao")  
  .addEventListener(  
    "change",  
    renderizarGestores  
  );
```

```
elemento("limparFiltros")  
  .addEventListener(  
    "click",  
    () => {  
  
      elemento("buscarId").value = "";  
  
      elemento("filtroStatus").value = "";  
  
      elemento("filtroAtuacao").value = "";  
  
      renderizarGestores();  
  
    }  
  );
```

```
elemento("modalGestor")  
  .addEventListener(  
    "click",  
    evento => {  
  
      if (  
        evento.target ===  
        elemento("modalGestor")  
      ) {  
        fecharModal();  
      }  
    }  
  );
```

```
}
```

```
}
```

```
);
```

```
if (usuarioAtual) {  
    entrarSistema();  
}
```